

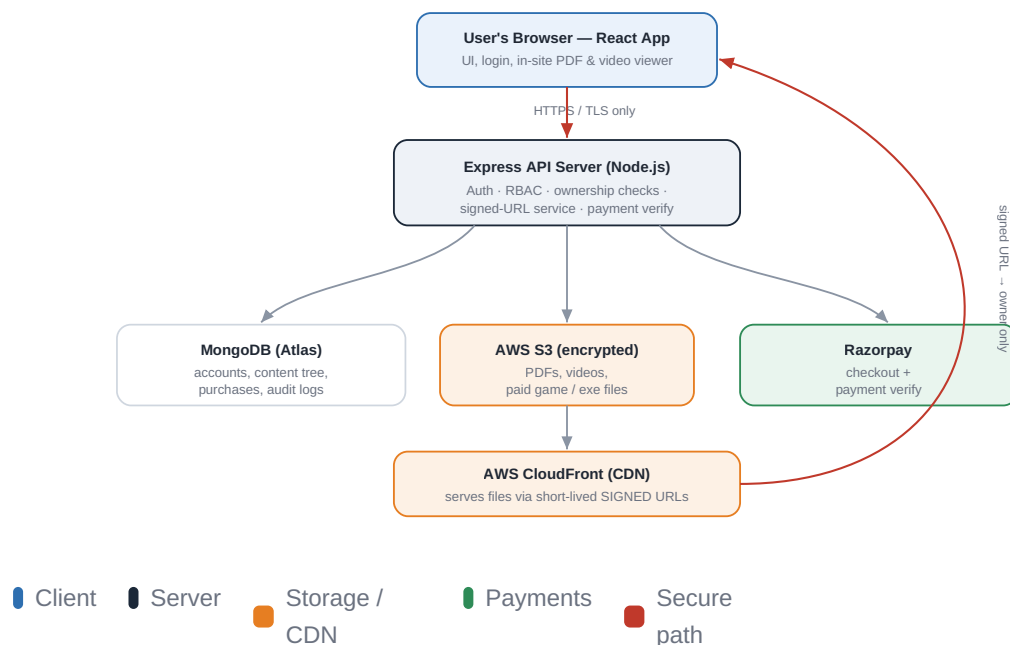
Vigno Smart Class — System Design & Architecture

Visual blueprint: architecture, data tree, content rules & ownership verification

Stack: MERN · AWS S3 + CloudFront · Razorpay · Steam-style protection (web-adapted)

1. High-Level Architecture

How the browser, server, database, storage, CDN and payments connect. The Express server is the single gatekeeper — the browser never touches files or the database directly.



Plain meaning: every request goes through the server. It decides **who you are** and **what you own**, then hands back a temporary link. There is no public, permanent address for any file.

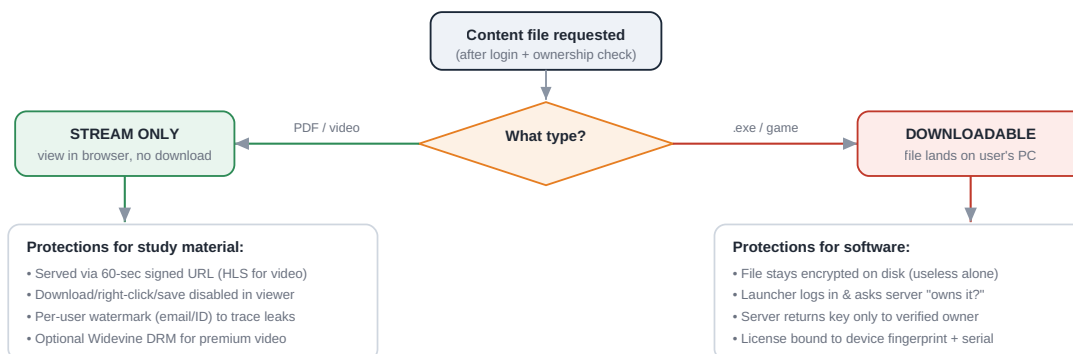
2. Content Hierarchy (Folder Tree)

One simple tree. The admin can create unlimited Modules, Chapters and Files. Files are free or paid.

```
Board (e.g., CBSE)
├── Class (Std_1 ... Std_12)
│   ├── Subject (Maths, Science ...) ← numbered sections on screen
│   │   ├── Module (folder card) ← admin makes unlimited
│   │   │   ├── Chapter / sub-folder ← admin makes unlimited
│   │   │   └── Content file
│   │       ├── study material (PDF, video) → stream-
│   │       └── software (.exe, game) → downloadable
only, view on site
(see §3 & §8)
```

3. Content Access Rules — Stream vs Download

New rule baked into the design: study material is **never downloaded** — it is only viewed on the website. Only software files (.exe, games) are downloaded to the user's machine. Each lane is protected differently.



Honest limit: once a download lands on a PC it cannot be policed by a website alone — that's why the encrypted-file + launcher model (a small desktop app) is the only way to truly verify a downloaded file on the local machine. Study material avoids the problem entirely by never leaving the browser.

4. Data Model (MongoDB Collections)

Each level of the tree is a collection. A child stores its parent's id, so the tree is fast to read.

Collection	Key fields	Role
users	name, email, passwordHash, role, isVerified, twoFAEnabled	Accounts + roles
boards → classes → subjects	name, parentId, order	Top of the tree
modules → chapters	title, parentId, icon, order	Folder cards & sub-folders
contents	title, chapterId, type (pdf/video = stream · exe/game = download) , s3Key, isPaid, price	The actual files
purchases	userId, contentId, amount, razorpayPaymentId, status	Ownership record
licenses	userId, contentId, deviceFingerprint, serial, issuedAt	For downloadable software only
favorites	userId, contentId	"Favs" tab
auditlogs	actorId, action, targetId, ip, time	Security trail

5. Login & Account Flow

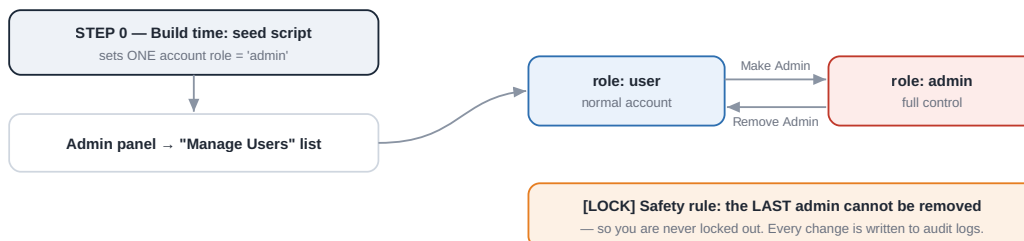
One login page for everyone — no separate admin button. After login, the user's **role** decides what they can do.



The JWT lives in an **httpOnly + Secure + SameSite** cookie, so scripts/attackers can't read it. A short access token + longer refresh token keeps users signed in safely.

6. Admin Roles & Management

First admin is set once during building. After that, admins can promote/demote any logged-in user — with a guard so the last admin can never be removed.

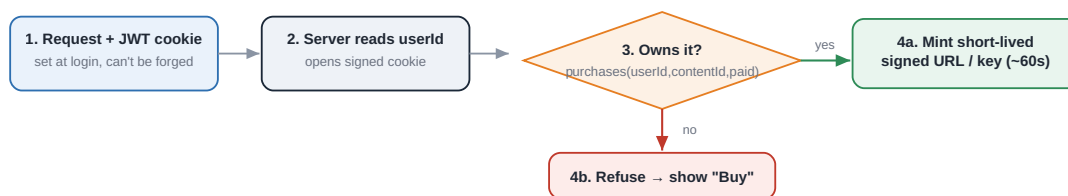


7. How Ownership Is Verified — The Account, Not the File

The single most important rule of web file protection: the file on disk proves nothing. Ownership lives on the server, in the purchases collection, and is re-checked **before every delivery**.

The match is: verified identity (JWT) ↔ ownership record (purchases) ↔ permission to unlock. The file never self-verifies — the logged-in account does, against the database.

The matching process (step by step)



What this means for the two content types

Type	After ownership check passes...	Why it's safe
Study material (PDF, video)	Server streams it via a 60-second signed URL; nothing usable is saved to disk.	No permanent file exists to copy; a shared link is already expired. Closest web version of Steam's "decrypt in memory only."
Software (.exe, game)	The encrypted file downloads; a launcher later asks the server "owns it?" and only then receives the decryption key.	The raw file on disk is useless without the key, and the key is bound to the account + device.

The honest limit (be clear about this)

A website **cannot** verify a raw file that's already downloaded and decrypted on someone's computer — browsers deliberately can't reach into a machine like that. So study material is kept **stream-only** to avoid the problem, and downloadable software needs the **encrypted-file + launcher** approach (a small separate desktop app) for true on-device verification. Watermarking remains the practical deterrent: even a leaked file can be traced back to the buyer.

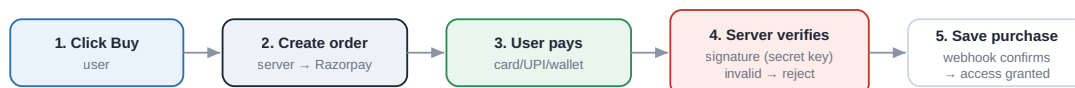
8. File Security — Steam Idea → Web Equivalent

Same idea as Steam: lock the file, verify the owner, then unlock briefly. Rebuilt with web tools.

Steam does	We do on the web	Tech
Ships game encrypted (CEG)	Files encrypted at rest	S3 SSE-KMS
Checks login first	Require JWT before any file request	JWT + cookies
"Do you own this?"	Check purchases before serving	Express middleware + MongoDB
Key only to owner	Short-lived signed URL / key to owner	CloudFront signed URLs
Decrypt in memory only	Study material: stream, link expires fast	Time-limited URLs + HLS
Client verifies at launch	Software: launcher app re-checks ownership before unlocking	Desktop launcher + license/device binding
Anti-tamper / trace	Per-user watermark on PDFs/videos; optional video DRM	Watermarking · Widevine (option)

9. Payment Flow (Razorpay)

A two-step verify means nobody can fake a payment — only our server holds the secret key needed to confirm it.



10. Deployment & Infrastructure

Where each part lives in production, and the security/speed layers around it.